



Govt. College UNIVERSITY, FAISALABAD

External Semester Examinations Fall-2023-2024

Roll No.: _____

Degree: BS-CS

Semester: 7th

Marks: 80

Part: Subjective

Course Title: Mobile & Application Development

Course Code: CSI-

Note: Attempt all questions. All the questions carry equal marks.

Time allowed: 2:30

Question # 2:

Briefly discuss the layered architecture of Android Operating system with a diagram.

Question # 3:

What is an activity and activity life cycle? Explain activity life cycle using a flow chart.

Question # 4:

- Describe the event handling in Android.
- Write a program in Kotlin or Java language to find the power of a number. The base and power values will be scanned from keyboard.

Question # 5:

Describe the difference between Explicit and Implicit Intents in Android. Also provide example of Explicit Intent with and without data provision and example of Implicit Intent as opening a web page <https://www.gcuf.edu.pk> with codes.

→ For running Java program, a computer needs JRE.

lecture NO: 07

Android Architecture:

interact with hardware

divide into Five stages:

- ① Linux kernel
- ② Libraries
- ③ Android run time
- ④ Application framework
- ⑤ Application.

linux kernel:

(2 to Part last)

- heart of operating system.
- contains essential hardware (drivers has cheez ko alg alg koty hn (means functionality)) like audio, camera, bluetooth etc.
- Provides Functionalities such as:
 - power mangment, process mangment,
 - memory mangment.
- Provide level of abstraction b/w device hardware and upper layers of android.

Libraries:

(upper linux & kernel)

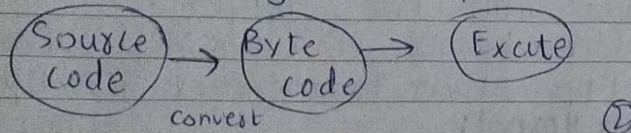
- linux kernel libraries:

NAB

SQLite Web-kit SSL OpenGL
 → provide various classes used for database management.
 → it is web browser engine used to display internet content.
 → Secure socket layer. provide internet security. For 3D convert types text. graphics
 → Java intecolave

Android runtime:

- provide core libraries and virtual machine (DVM).
- These core libraries enabled android developers to write android applications using standard Java programming language.
- DVM major component of Android OS same like JVM.
- responsible for running android appli. takes less memory and provide fast memory.



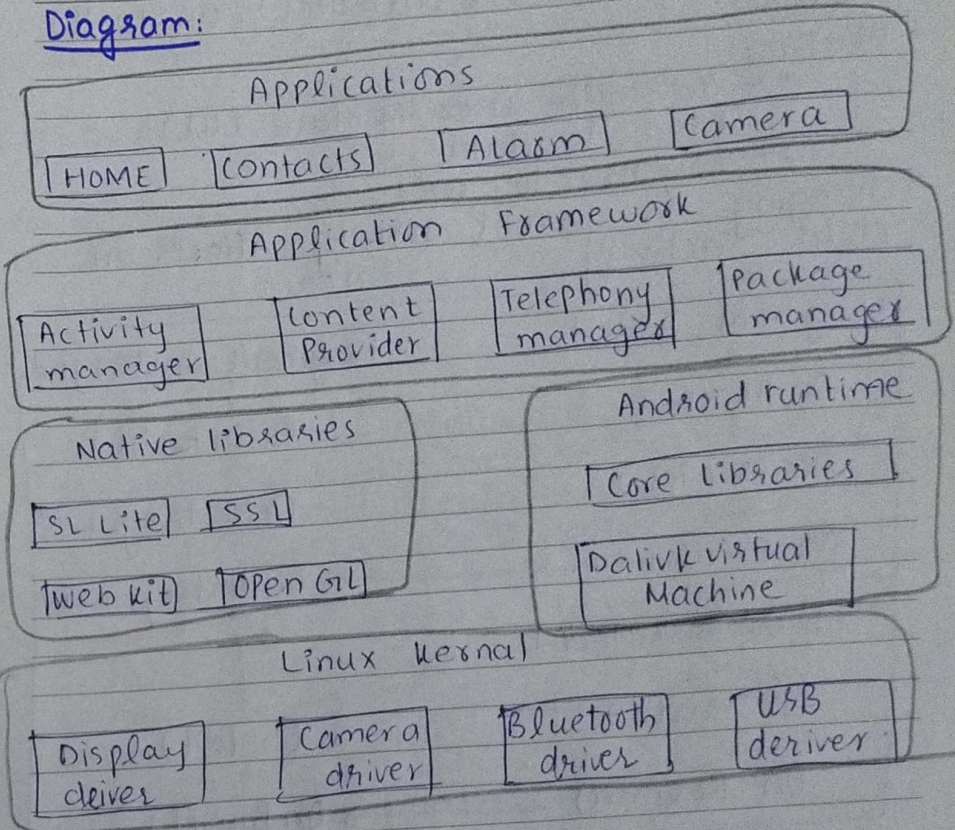
Application Framework:

- provides resources, content providers, activity managers, telephony managers etc
- API's used for android application developer.

Application:

- present at top. created by developers like games, contacts, browsers are installed on this layer.

Diagram:



SLIDE NO: 06

Fragment life cycle:

- refers to series of events that a Fragment goes through from its creation to its destruction.

example: whatsapp > chats > stats > contacts.

Android Activity Lifecycle



In **Android**, an **activity** is referred to as one screen in an application. It is very similar to a single window of any desktop application. An Android app consists of one or more screens or activities.

Each activity goes through various stages or a lifecycle and is managed by activity stacks. So when a new activity starts, the previous one always remains below it.

There are four stages of an activity.

- If an activity is in the foreground of the screen at the top of the stack, then it is said to be active or running. This is usually the activity that the user is currently interacting with.
- If an activity has lost focus and a non-full-sized or transparent activity has focused on top of your activity. In such a case either another activity has a higher position in multi-window mode or the activity itself is not focusable in the current window mode. Such activity is completely alive.

Android Activity Lifecycle

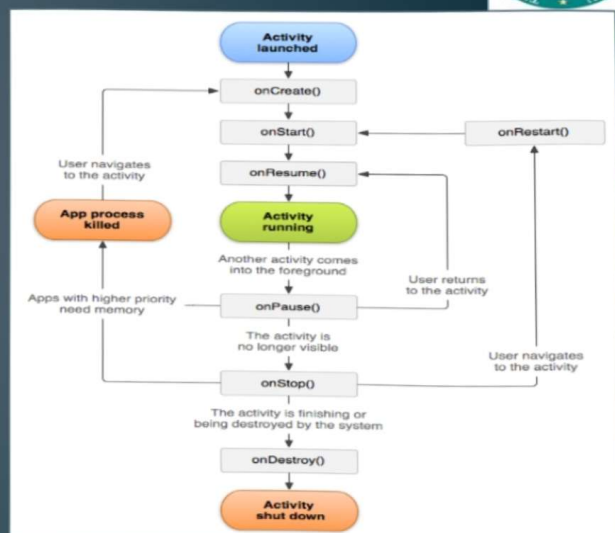


- If an activity is completely hidden by another activity, it is stopped or hidden. It still retains all the information, and as its window is hidden thus it will often be killed by the system when memory is needed elsewhere.
- The system can destroy the activity from memory by either asking it to finish or simply killing its process. When it is displayed again to the user, it must be completely restarted and restored to its previous state.
- For each stage, android provides us with a set of 7 methods that have their own significance for each stage in the life cycle.

Android Activity Lifecycle



Method	Description
onCreate	called when activity is first created.
onStart	called when activity is becoming visible to the user.
onResume	called when activity will start interacting with the user.
onPause	called when activity is not visible to the user.
onStop	called when activity is no longer visible to the user.
onRestart	called after your activity is stopped, prior to start.
onDestroy	called before the activity is destroyed.



Android Activity Lifecycle Example



```
1. File: activity_main.xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context="example.javatpoint.com.activitylifecycle.MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!" />

</LinearLayout>
```

Android Activity Lifecycle Example



```
File: MainActivity.java
package example.javatpoint.com.activitylifecycle;

import android.app.Activity;
import android.os.Bundle;
import android.util.Log;

public class MainActivity extends Activity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Log.d("lifecycle", "onCreate invoked");
    }
}
```

Android Activity Lifecycle Example



OnStart

```
@Override
protected void onStart() {
    super.onStart();
    Log.d("lifecycle", "onStart invoked");
}
```

OnPause

```
@Override
protected void onPause() {
    super.onPause();
    Log.d("lifecycle", "onPause invoked");
}
```

OnResume

```
@Override
protected void onResume() {
    super.onResume();
    Log.d("lifecycle", "onResume invoked");
}
```

OnStop

```
@Override
protected void onStop() {
    super.onStop();
    Log.d("lifecycle", "onStop invoked");
}
```

Android Activity Lifecycle Example



OnRestart

```
@Override
protected void onRestart() {
    super.onRestart();
    Log.d("lifecycle", "onRestart invoked");
}
```

OnDestroy

```
@Override
protected void onDestroy() {
    super.onDestroy();
    Log.d("lifecycle", "onDestroy invoked");
}
```

Question #4:

a) Describe the event handling in Android.

Event handling in Android involves responding to user interactions like clicks, touches, or key presses. The primary mechanisms are:

1. Event Listeners:

- Interfaces that listen for specific events.
- Examples: `OnClickListener`, `OnTouchListener`, `OnKeyListener`.
- Example code:

```
java Copy
button.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        // Handle click event
    }
});
```

2. Event Handlers:

- Methods that handle events.
- Example: `onClick()`, `onTouch()`, `onKey()`.

3. Touch Mode:

- Android devices support touch events, which are handled using `MotionEvent`.

b) Write a program in Kotlin or Java language to find the power of a number. The base and power values will be scanned from the keyboard.

Kotlin Code:

```
kotlin Copy
import java.util.Scanner

fun main() {
    val scanner = Scanner(System.`in`)
    print("Enter base: ")
    val base = scanner.nextInt()
    print("Enter power: ")
    val power = scanner.nextInt()

    val result = Math.pow(base.toDouble(), power.toDouble())
    println("Result: $result")
}
```

Java Code:

```
java Copy
import java.util.Scanner;

public class PowerCalculation {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter base: ");
        int base = scanner.nextInt();
        System.out.print("Enter power: ");
        int power = scanner.nextInt();

        double result = Math.pow(base, power);
        System.out.println("Result: " + result);
    }
}
```

Question #5: Describe the difference between Explicit and Implicit Intents in Android.

Question #5: Describe the difference between Explicit and Implicit Intents in Android. Also, provide an example of Explicit Intent without data provision and an example of Implicit Intent to open a web page.

Explicit Intent:

- Used to launch a specific component within the app (e. g. , another activity).
- Specifies the exact class to be invoked.
- Example without data:

java

Copy

```
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
startActivity(intent);
```

Implicit Intent:

- Used to request an action from another app or component.
- Does not specify the exact component; the system resolves it.
- Example to open a web page:

java

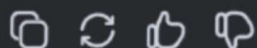
Copy

```
Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse("https://www.cu
t.edu.th"));
startActivity(intent);
```

Difference:

- Explicit Intent: Targets a specific component.
- Implicit Intent: Declares a general action for the system to handle.

Let me know if you need further clarification!



 New chat

Message DeepSeek

